

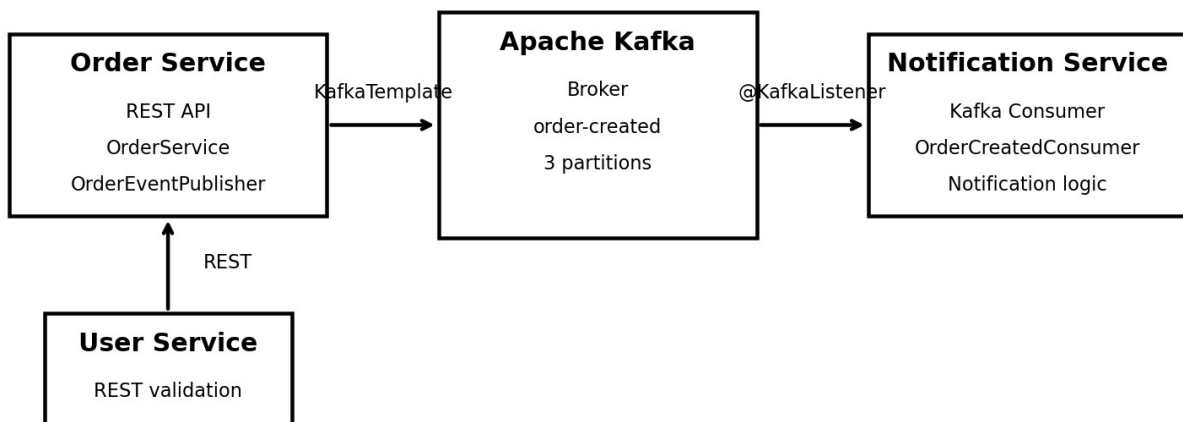
JIRA TASK 04

Asynchronous Communication with Apache Kafka

Implementation, Testing and Presentation Documentation

Task 04 — Asynchronous Order Event Flow

Order Service publishes → Kafka stores → Notification Service consumes



Prepared and explained by Abhishek
24 September 2026

Contents

- 1. Task 04 — What I Had to Build
- 2. Kafka Concepts Explained Simply
- 3. What I Built From Scratch
- 4. Project Architecture
- 5. Step-by-Step Implementation Workflow
- 6. Files Added or Modified and Why
- 7. Event Model and Message Flow
- 8. Testing and Evidence
- 9. Failure and Recovery Test
- 10. Why Docker Desktop Was Used
- 11. Final Task 04 Checklist
- 12. Presentation / Demo Flow
- 13. Final Summary

1. Task 04 — What I Had to Build

In Task 04, I implemented asynchronous communication between my microservices using Apache Kafka. The goal was to understand how one service can publish an event without directly waiting for another service to finish its work.

The required business flow was:

Order Service → Kafka → Notification Service

- Understand message, event, producer, consumer, topic, partition, consumer group and offset.
- Understand synchronous versus asynchronous communication.
- Set up Kafka locally and verify that it runs.
- Create the order-created topic.
- Make Order Service publish an ORDER_CREATED event.
- Make Notification Service consume the event.
- Test normal publishing and consuming.
- Stop Notification Service, create an order, restart it and verify that the pending event is consumed.

2. Kafka Concepts Explained Simply

I learned these concepts before writing the Kafka code. I explain them in simple language so that a beginner can follow the idea.

Concept	Simple explanation
Message	A piece of information sent from one application to another.
Event	A message that says something happened, such as

	ORDER_CREATED.
Producer	The application that sends an event. Here, Order Service.
Consumer	The application that reads an event. Here, Notification Service.
Topic	A named place in Kafka where related events are stored. Here, order-created.
Partition	A topic can be divided into ordered sections. Our topic has 3 partitions.
Consumer group	A group of consumers that work together to read a topic. Here, notification-service.
Offset	A position that helps Kafka know which messages a consumer has already processed.

Synchronous vs asynchronous

Synchronous: Order Service → Notification Service → response

The first service waits for the second service.

Asynchronous: Order Service → Kafka → Notification Service

The producer publishes an event. The consumer can process it independently.

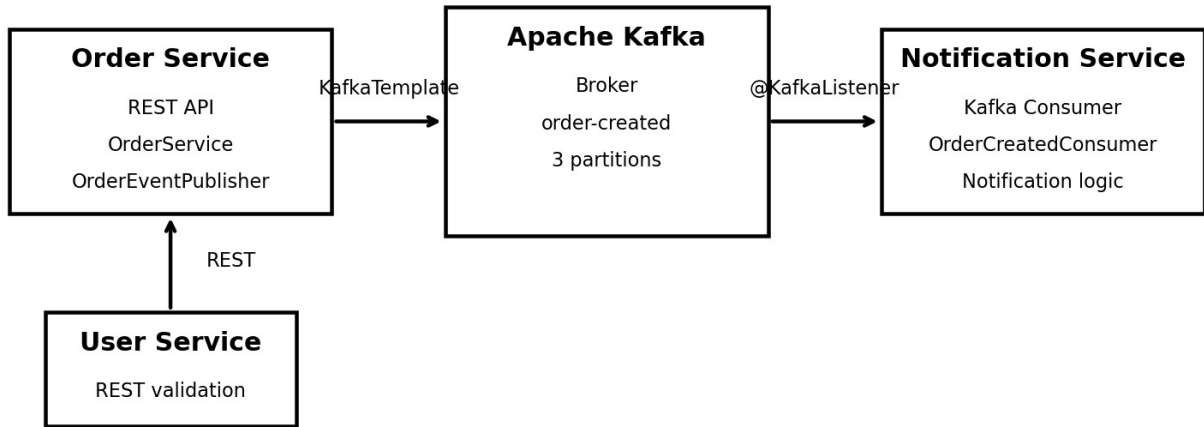
3. What I Built From Scratch

1. I installed Docker Desktop and configured WSL 2 so I could run Kafka locally.
2. I created a separate infrastructure/Kafka area for Kafka configuration.
3. I created a single-node Apache Kafka broker using KRaft mode.
4. I created the order-created topic with 3 partitions and replication factor 1 for local development.
5. I added Spring Kafka support to Order Service and Notification Service.
6. I configured both services to connect to Kafka through a configurable bootstrap-server property.
7. I created OrderCreatedEvent models for the event contract.
8. I created OrderEventPublisher in Order Service.
9. I updated OrderService to publish an event after an order is saved.
10. I created OrderCreatedConsumer in Notification Service.
11. I tested normal communication and then tested consumer failure and recovery.

4. Project Architecture

Task 04 — Asynchronous Order Event Flow

Order Service publishes → Kafka stores → Notification Service consumes



The existing REST communication with User Service remains. Kafka is an additional asynchronous path for order-created events.

Order creation workflow

12. Postman sends POST /api/v1/orders to Order Service.
13. Order Service validates the user using the existing User Service REST call.
14. Order Service saves the order in MySQL.
15. Order Service creates an OrderCreatedEvent.
16. OrderEventPublisher sends it to the order-created Kafka topic.
17. Kafka stores the event.
18. Notification Service receives it using @KafkaListener.
19. Notification Service processes the event and logs the notification requirement.

5. Step-by-Step Implementation Workflow

Step 1 — Kafka infrastructure

I created infrastructure/Kafka/docker-compose.yml and used apache/kafka:4.1.1. Kafka runs as a single-node KRaft broker for local development.

Kafka is exposed to the Spring Boot applications at localhost:9092.

Step 2 — Kafka topic

I created the topic order-created with 3 partitions and replication factor 1.

The three partitions allow Kafka to distribute messages. Replication factor 1 is suitable for the single-broker local setup used for this task.

Step 3 — Spring Kafka dependency

I added spring-boot-starter-kafka to both services. This provides the Spring Kafka integration used by KafkaTemplate and @KafkaListener.

Step 4 — Kafka configuration

Order Service application.yml uses an environment-variable-friendly broker setting and JSON serialization:

```
spring:  
kafka:  
bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}  
producer:  
key-serializer: org.apache.kafka.common.serialization.StringSerializer  
value-serializer:  
org.springframework.kafka.support.serializer.JsonSerializer
```

Step 5 — Event model

The event carries the minimum information needed by the notification flow:

```
{  
  "orderId": 10,  
  "userId": 1,  
  "amount": 2500.0,  
  "eventType": "ORDER_CREATED",  
  "createdAt": "2026-09-24T12:33:14..."  
}
```

Step 6 — Producer

OrderEventPublisher uses KafkaTemplate to send the event to order-created. I use orderId as the Kafka message key.

```
kafkaTemplate.send("order-created", event.getOrderId().toString(), event);
```

Step 7 — Connect producer to order creation

After orderRepository.save(order), OrderService creates the event and calls orderEventPublisher.publishOrderCreated(event). This keeps Kafka publishing out of the controller.

Step 8 — Consumer

Notification Service uses OrderCreatedConsumer with a Kafka listener and the consumer group notification-service.

@KafkaListener(topics = "order-created", groupId = "notification-service")

Step 9 — Normal test

I created an order from Postman and verified the 201 Created response and the Notification Service output.

6. Files Added or Modified and Why

File	Change	Why
infrastructure/Kafka/docker-compose.yml	Added	Runs the local Kafka broker in Docker.
order-service/pom.xml	Modified	Adds Spring Kafka support.
order-service/src/main/resources/application.yml	Modified	Configures the Kafka broker and JSON producer.
order-service/.../event/OrderCreatedEvent.java	Added	Defines the ORDER_CREATED event.
order-service/.../event/OrderEventPublisher.java	Added	Publishes the event to Kafka.
order-service/.../service/OrderService.java	Modified	Creates and publishes the event after saving the order.
notification-service/pom.xml	Modified	Adds Spring Kafka support.
notification-service/src/main/resources/application.properties	Modified	Configures the Kafka broker connection.
notification-service/.../event/OrderCreatedEvent.java	Added	Defines the event structure expected by the consumer.
notification-service/.../consumer/OrderCreatedConsumer.java	Added	Consumes and processes order-created events.

I kept the producer and consumer event classes inside their own services. This avoids making the two microservices depend on one shared Java class and keeps them independently deployable.

7. Event Model and Message Flow

The event contains:

- orderId — identifies the order.
- userId — identifies the user.
- amount — carries the order amount.
- eventType — tells the consumer what happened.
- createdAt — records when the event was created.

Example event:

{"orderId":10,"userId":1,"amount":2500.0,"eventType":"ORDER_CREATED","createdAt":"..."}

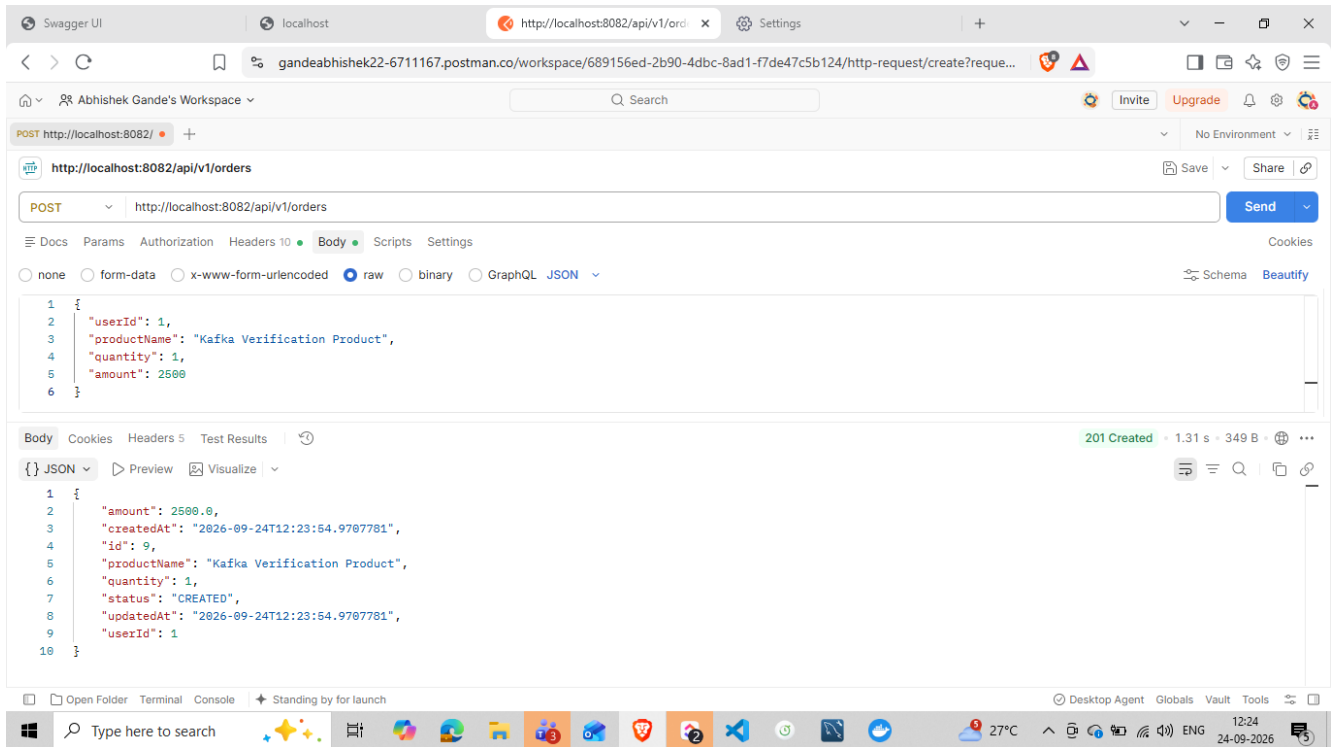
The order ID is used as the Kafka message key. This gives the event a stable key and is useful when messages for the same order need consistent partitioning.

8. Testing and Evidence

I tested the system using Postman, the Spring Boot terminals and Kafka command-line tools.

Test 1 — Create an order

A POST request to `http://localhost:8082/api/v1/orders` created Order 9 and returned HTTP 201 Created.

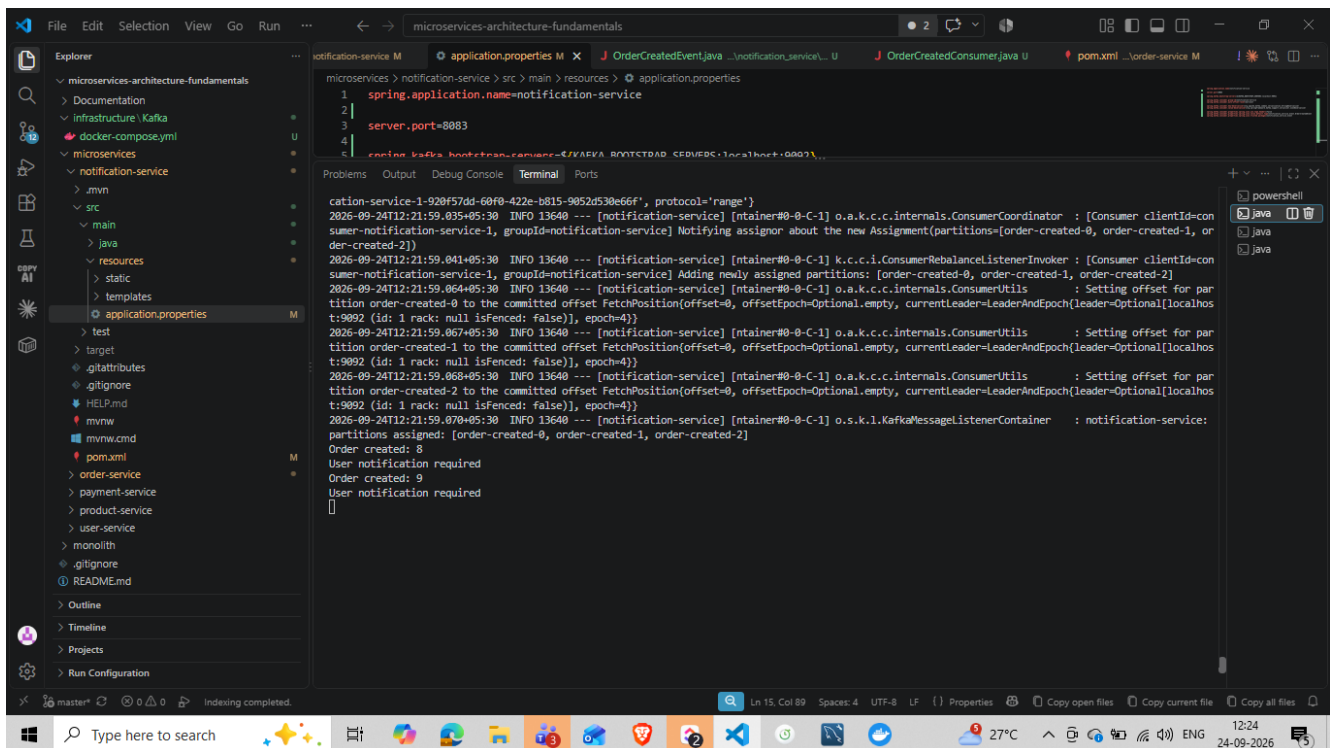


Evidence: Postman — Order 9 created successfully.

Test 2 — Notification Service receives the event

The Notification Service terminal showed:

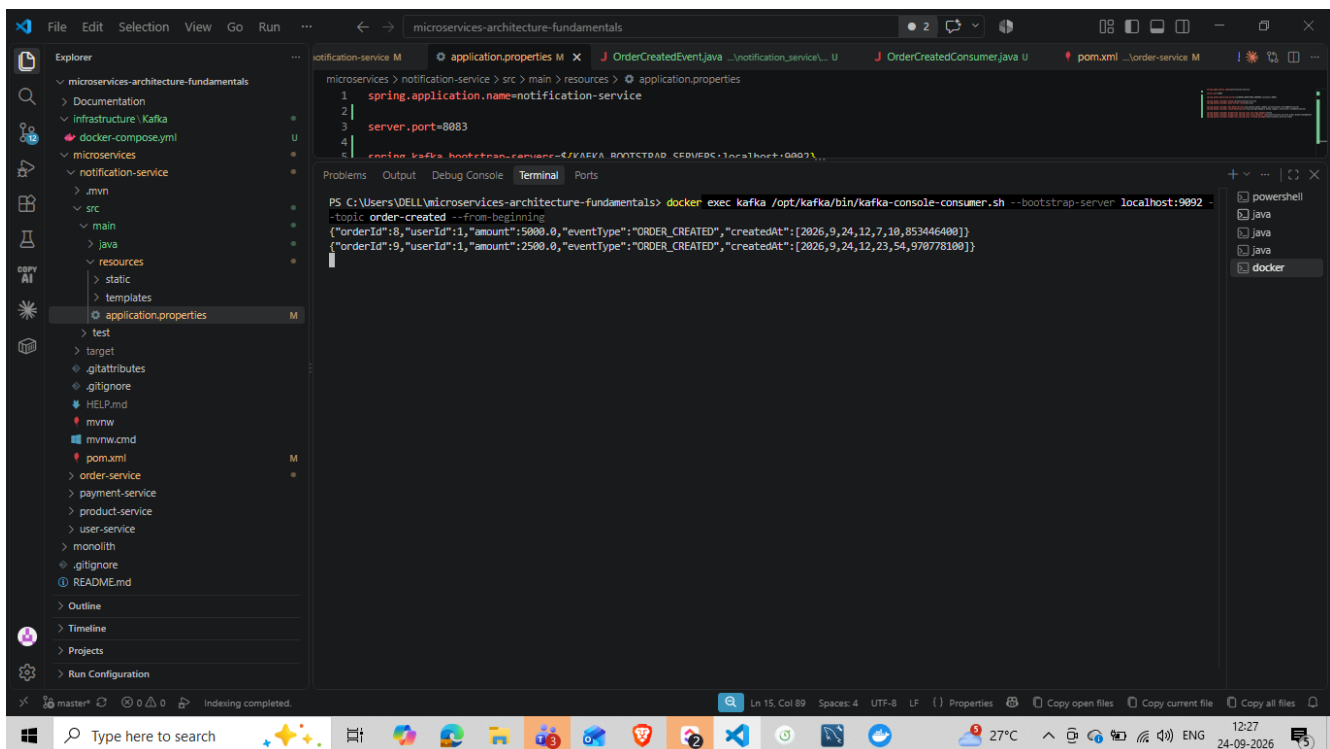
Order created: 9
User notification required



Evidence: Notification Service consumed Order 9.

Test 3 — Verify the Kafka topic directly

I ran a Kafka console consumer and confirmed that the order-created topic contained ORDER_CREATED JSON events for Orders 8 and 9.



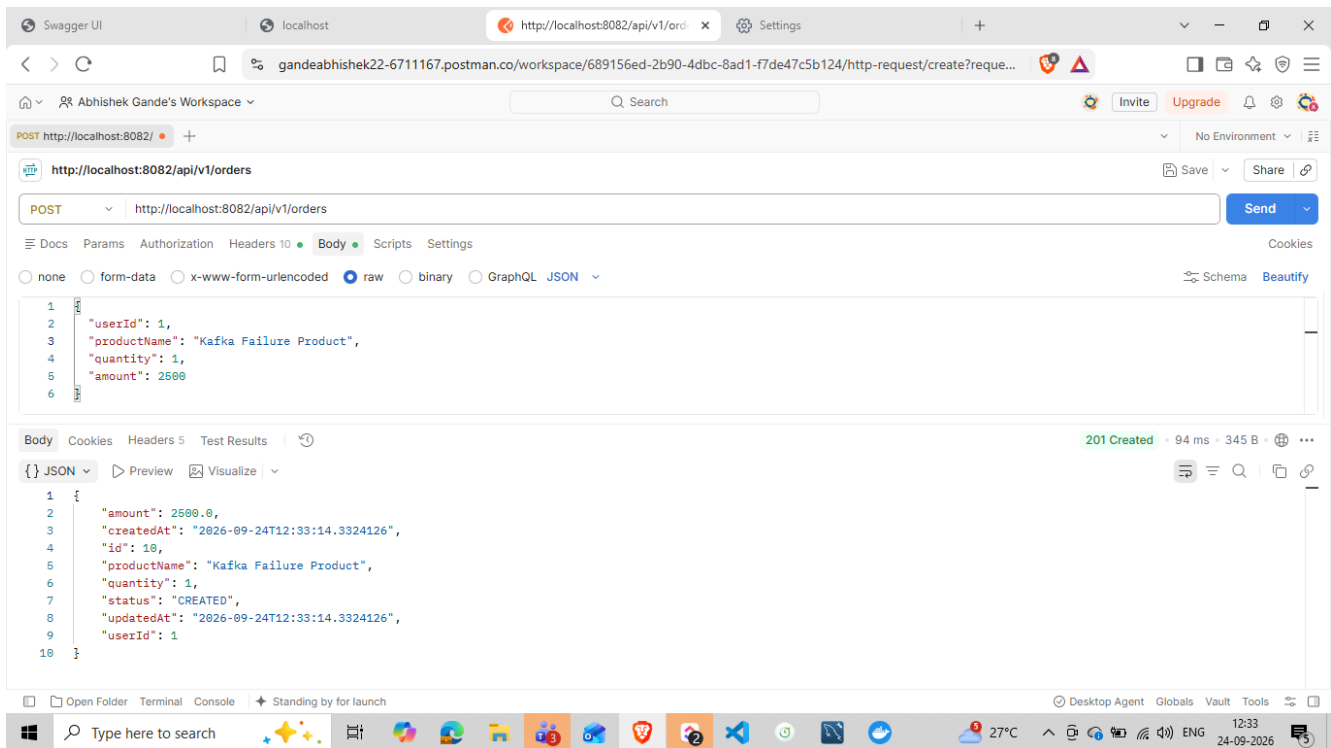
Evidence: Kafka topic contains the published events.

9. Failure and Recovery Test

This test demonstrates why asynchronous messaging is useful when the consumer is temporarily unavailable.

20. I stopped Notification Service only.
21. I kept Kafka and Order Service running.
22. I created a new order from Postman.
23. Order Service returned HTTP 201 Created with Order ID 10.
24. Notification Service could not consume the event while it was stopped.
25. I restarted Notification Service.
26. The consumer rejoined the notification-service group and Kafka delivered the pending event.
27. The terminal showed: Order created: 10 and User notification required.

Postman evidence for Order 10:



Notification Service evidence after restart:

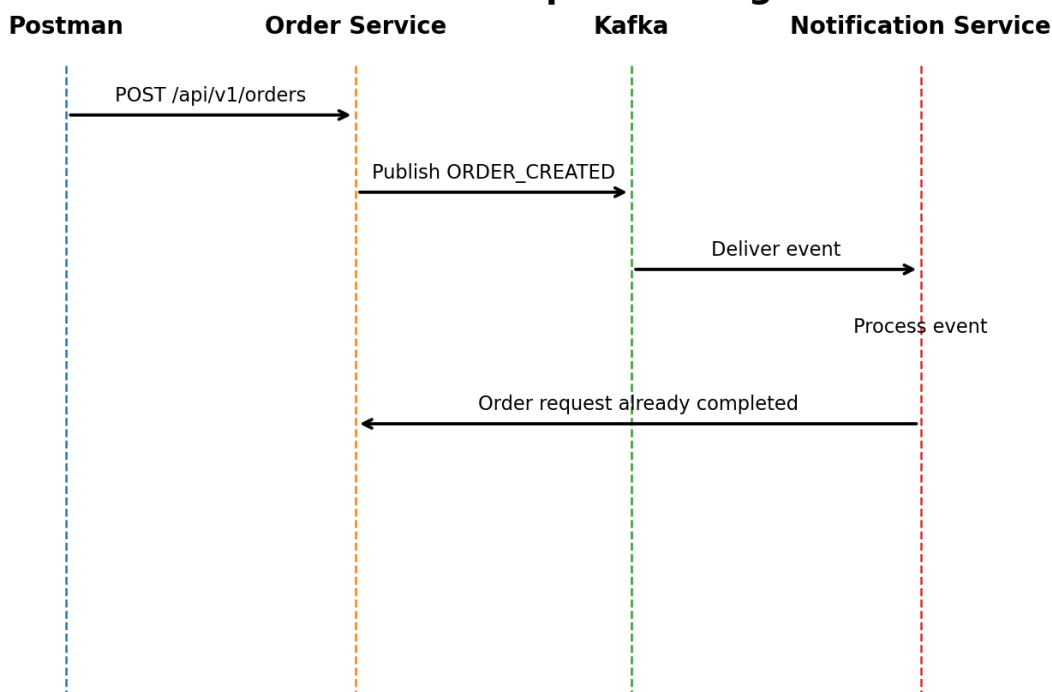
The screenshot shows an IDE with the following components:

- Explorer:** A file tree on the left showing the project structure, including `microservices-architecture-fundamentals`, `notification-service`, and `application.properties`.
- application.properties:** The file is open in the editor, showing properties like `spring.application.name=notification-service` and `server.port=8083`.
- Terminal:** A terminal window at the bottom displays Kafka logs. The logs show the `notification-service` consumer joining a group, successfully joining the group, and then processing the `ORDER_CREATED` event. The logs also show the consumer's internal state, including the current leader and epoch.

Result: the event was retained by Kafka and consumed after the Notification Service restarted.

10. Sequence Diagram

Task 04 — Sequence Diagram



Failure test: consumer stopped → Order 10 created → consumer restarted → pending event consumed

The sequence shows that Order Service publishes to Kafka instead of directly calling Notification Service for the notification event.

11. Why Docker Desktop Was Used

Docker Desktop provides the environment where Kafka actually runs. The terminal is only where I observe and control the system.

- Docker Desktop runs the Kafka container.
- Kafka runs separately from my Spring Boot applications.
- Spring Boot connects to Kafka at localhost:9092.
- The terminal is used to start, stop and inspect the applications and Kafka.
- Keeping Kafka in infrastructure/Kafka separates infrastructure from application code.

Simple picture:

Windows → Docker Desktop → Kafka container → order-created topic → Spring Boot services

12. Final Task 04 Checklist

Deliverable	Status
Kafka setup	Completed
Kafka topic: order-created	Completed
Producer	Completed
Consumer	Completed
Event model	Completed
Order event flow	Completed
Normal publishing test	Completed
Consumer test	Completed
Failure/recovery test	Completed
Architecture diagram	Included
Sequence diagram	Included
Postman test evidence	Included
Implementation documentation	Included

13. Presentation / Demo Flow

- 1. Problem:** I needed an asynchronous way for Order Service to announce that an order was created.
- 2. Solution:** I introduced Apache Kafka between the producer and consumer.
- 3. Infrastructure:** I run Kafka locally using Docker Desktop and Docker Compose.
- 4. Producer:** Order Service publishes an ORDER_CREATED event after saving an order.
- 5. Topic:** Kafka stores the event in order-created.

- 6. Consumer:** Notification Service listens to the topic using `@KafkaListener`.
- 7. Normal test:** Creating Order 9 produced an event and Notification Service consumed it.
- 8. Kafka verification:** I read the topic directly and confirmed the JSON event was stored.
- 9. Failure test:** I stopped Notification Service and created Order 10.
- 10. Recovery:** After restarting Notification Service, Kafka delivered Order 10.
- 11. Conclusion:** The required asynchronous event flow and failure/recovery behavior are working.

14. Final Summary — My Explanation

In Task 04, I implemented asynchronous communication using Apache Kafka. I first prepared a local Kafka environment with Docker Desktop and Docker Compose. I then created the order-created topic and connected my Spring Boot services to Kafka.

I kept the existing REST communication between Order Service and User Service because it is still needed for user validation. Separately, after an order is saved, Order Service publishes an `ORDER_CREATED` event to Kafka.

Notification Service does not need to be called directly by Order Service for this event. It listens to Kafka and processes the event when it is available. This makes the notification flow asynchronous.

Finally, I tested the failure case. I stopped Notification Service, created Order 10, restarted Notification Service and verified that Kafka delivered the pending event. This completed the main implementation and testing requirements of Task 04.

Task 04 — Asynchronous Communication with Apache Kafka

Implemented, tested and documented by Abhishek